

TP : Détection de discours haineux avec une régression logistique

Contexte d'entreprise fictif

Vous travaillez pour **ModeraTech**, une entreprise spécialisée dans la création de solutions d'intelligence artificielle pour modérer les plateformes de réseaux sociaux. Une grande entreprise française, **VidéoPartage**, souhaite intégrer un système automatique pour détecter et signaler les commentaires haineux sur ses vidéos.

Votre mission est de construire un modèle capable d'analyser des textes en français et de classifier un commentaire comme **haineux** ou **non haineux**. Les données proviennent d'un échantillon synthétique créé par l'équipe de ModeraTech.

Objectifs pédagogiques

1. Comprendre le cycle de création d'un modèle de Machine Learning : préparation des données, entraînement, évaluation.
2. Travailler avec des données textuelles en français.
3. Appliquer une régression logistique pour un problème de classification binaire.

Étape 0 : Preparation

Vous aurez besoin de `python` et des packages suivants : `pandas` , `sklearn` et tout ca s'installe avec `pip install XXX`

Étape 1 : Données d'entrée

Voici un jeu de données fictif :

```
# Création des données fictives
import pandas as pd

# Dataset synthétique
data = {
    "text": [
        "Je te déteste, tu es horrible !", # Haineux
        "J'aime beaucoup cette vidéo, merci.", # Non haineux
        "Va te faire voir, imbécile.", # Haineux
        "Quel contenu inspirant, bravo à l'équipe !", # Non haineux
        "Tu es vraiment nul et inutile.", # Haineux
        "Je suis impressionné par la qualité de cette vidéo.", # Non haineux
        "Ferme-la, personne ne veut entendre ça.", # Haineux
        "C'est une discussion constructive, merci pour vos efforts.", # Non haineux
        "Ce commentaire est complètement stupide et inutile.", # Haineux
        "Merci pour cette vidéo, elle m'a beaucoup aidé !", # Non haineux
        "Personne n'a besoin de voir des bêtises pareilles.", # Haineux
        "Excellent contenu, continuez comme ça !", # Non haineux
        "Tu ne comprends rien, arrête de commenter.", # Haineux
    ]
}
```

```

    "Bravo, c'est exactement ce que je cherchais.", # Non haineux
    "Espèce d'idiot, tu ne sais même pas de quoi tu parles.", # Haineux
    "Cette vidéo est très claire, merci pour le travail.", # Non haineux
    "Tu es une honte, personne ne veut lire ça.", # Haineux
    "Le tutoriel est super bien expliqué, merci !", # Non haineux
    "C'est complètement débile, arrête de poster.", # Haineux
    "J'adore cette chaîne, toujours des vidéos intéressantes.", # Non haineux
    "Dégage d'ici, personne ne te supporte.", # Haineux
    "Merci pour ces conseils, c'est vraiment utile.", # Non haineux
    "T'es vraiment le pire, tes vidéos sont nulles.", # Haineux
    "Une très bonne vidéo, claire et précise, bravo !", # Non haineux
],
"label": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}

# Conversion en DataFrame
df = pd.DataFrame(data)
print(df)

```

Étape 2 : Prétraitement des données

Nettoyez les textes (supprimer les caractères spéciaux, mettre en minuscule, supprimer les mots vides).

```

import re
from sklearn.feature_extraction.text import CountVectorizer

# Fonction de nettoyage
def clean_text(text):
    text = text.lower() # Mettre en minuscule
    text = re.sub(r'[\W\s]', '', text) # Supprimer les caractères spéciaux
    return text

# Appliquer le nettoyage
df['text_clean'] = df['text'].apply(clean_text)

french_stopwords = [
    "le", "la", "les", "un", "une", "des", "du", "de", "dans", "et", "en", "au",
    "aux", "avec", "ce", "ces", "pour", "par", "sur", "pas", "plus", "où", "mais",
    "ou", "donc", "ni", "car", "ne", "que", "qui", "quoi", "quand", "à", "son",
    "sa", "ses", "ils", "elles", "nous", "vous", "est", "sont", "cette", "cet",
    "aussi", "être", "avoir", "faire", "comme", "tout", "bien", "mal", "on", "lui"
]

# Vectorisation (bag of words)
vectorizer = CountVectorizer(stop_words=french_stopwords, max_features=100)
X = vectorizer.fit_transform(df['text_clean'])
y = df['label']

print("Vectorisation terminée.")

```

Étape 3 : Division des données et entraînement du modèle

Divisez les données en ensembles d'entraînement et de test, puis entraînez une régression logistique.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Division des données
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Entraînement du modèle
model = LogisticRegression()
model.fit(X_train, y_train)

print("Modèle entraîné avec succès.")
```

Étape 4 : Évaluation des performances

Évaluez le modèle à l'aide des métriques standard et affichez les résultats.

```
from sklearn.metrics import classification_report, confusion_matrix

# Prédictions
y_pred = model.predict(X_test)

# Rapport de classification
print("Rapport de classification :")
print(classification_report(y_test, y_pred))

# Matrice de confusion
print("Matrice de confusion :")
print(confusion_matrix(y_test, y_pred))
```

Étape 5 : Tester le modèle sur de nouveaux commentaires

Ajoutez des commentaires fictifs pour tester le modèle.

```
# Nouvelles données
new_comments = [
    "Je ne supporte pas cette personne.", # Haineux
    "Cette vidéo est incroyable, merci pour votre travail.", # Non haineux
    "Arrête de dire n'importe quoi, imbécile.", # Haineux
    "Une excellente présentation, bravo à toute l'équipe." # Non haineux
]

# Nettoyage et vectorisation
new_comments_clean = [clean_text(comment) for comment in new_comments]
new_comments_vectorized = vectorizer.transform(new_comments_clean)
```

```
# Prédiction
predictions = model.predict(new_comments_vectorized)
for comment, label in zip(new_comments, predictions):
    print(f"Commentaire : '{comment}' -> {'Haineux' if label == 1 else 'Non haineux'}")
```

Analyse et discussion

1. Quels sont les résultats obtenus? Le modèle est-il performant?
 2. Quelles pourraient être les limites de ce modèle?
 3. Comment améliorer les performances? (Exemple : ajouter des données, utiliser des modèles plus avancés comme des transformers)
-

Livrables attendus

1. Code complet du TP avec commentaires.
2. Rapport d'analyse des performances du modèle.
3. Suggestions d'amélioration pour le modèle.